1. XOR each character with 0 — C

CODE

```
#include <stdio.h>
int main(void) {
   char *str = "Hello World";
   printf("Original String:%s\n", str);
   printf("After XOR with 0:");
   for (int i=0; str[i]!='\0'; i++) printf("%c", str[i] ^ 0);
   return 0;
}
```

OUTPUT

```
Original String:Hello World
After XOR with 0:Hello World
```

2. AND, OR and XOR each character with 127 — C

CODE

```
#include <stdio.h>
int main(void) {
   char *str = "Hello World";
   printf("Original String:%s\n", str);
   printf("After AND with 127:");
   for (int i=0; str[i]!='\0'; i++) printf("%c", str[i] & 127);
   printf("\nAfter XOR with 127:");
   for (int i=0; str[i]!='\0'; i++) printf("%c", str[i] ^ 127);
   printf("\nAfter OR with 127:");
   for (int i=0; str[i]!='\0'; i++) printf("%c", str[i] | 127);
   return 0;
}
```

OUTPUT

```
Original String:Hello World
After AND with 127:Hello World
After XOR with 127: \x37 \x1A \x13 \x13 \x10 \x5F \x28 \x10 \x0D \x13 \x1B
After OR with 127: \x7F \x7F \x7F \x7F \x7F \x7F \x7F \x7F \x7F \x7F \x7F
```

NOTE: Display note: XOR/OR produce control bytes that are not printable text. They are shown as hexadecimal escape bytes here; the compiled C program emits the actual bytes.

3. Caesar Cipher — Java

CODE

```
import java.util.Scanner;
public class Caesar {
    static String encrypt(String s,int k) {
        StringBuilder r=new StringBuilder(); k=(k%26+26)%26;
        for(char c:s.toCharArray()) {
            if(c>='A'&&c<='Z') c=(char)('A'+(c-'A'+k)%26);
            else if(c>='a'&&c<='z') c=(char)('a'+(c-'a'+k)%26);
            r.append(c);
        } return r.toString();
    }
    static String decrypt(String s,int k) { return encrypt(s,-k); }
    public static void main(String[] a) {
        Scanner sc=new Scanner(System.in);
        System.out.print("enter plain text "); String s=sc.nextLine();
        System.out.print("enter key"); int k=sc.nextInt();
        String e=encrypt(s,k);
        System.out.print("encrypted text is "+e);
        System.out.print("decrypted text is "+decrypt(e,k));
        sc.close();
    }
}
```

OUTPUT

```
enter plain text Hello
enter key3
encrypted text is Khoordecrypted text is Hello
```

4. Playfair Cipher — Java

CODE

```
import java.util.Scanner;
public class Playfair {
   static char[][] m=new char[5][5]; static int[] row=new int[26],col=new int[26];
   static void create(String key) {
      boolean[] used=new boolean[26]; used['J'-'A']=true;
      String s=key.toUpperCase().replace('J','I')+"ABCDEFGHIKLMNOPQRSTUVWXYZ"; int n=0;
      for(char c:s.toCharArray()) if(c>='A'&&c<='Z'&&!used[c-'A']) {
         used[c-'A']=true; m[n/5][n%5]=c; row[c-'A']=n/5; col[c-'A']=n%5; n++;
      }
   }
   static String prepare(String s) {
      s=s.toUpperCase().replace('J','I'); StringBuilder r=new StringBuilder();
      for(int i=0;i<s.length();) {
         char a=s.charAt(i++); r.append(a);
         if(i==s.length()||a==s.charAt(i)) r.append('X'); else r.append(s.charAt(i++));
      } return r.toString();
   }
   static String process(String s,boolean enc) {
      StringBuilder r=new StringBuilder(); int d=enc?1:4;
      for(int i=0;i<s.length();i+=2) {
         char a=s.charAt(i),b=s.charAt(i+1);
         int x=row[a-'A'],y=row[b-'A'],p=col[a-'A'],q=col[b-'A'];
         if(x==y){p=(p+d)%5;q=(q+d)%5;}
         else if(p==q){x=(x+d)%5;y=(y+d)%5;}
         else {int t=p;p=q;q=t;}
         r.append(m[x][p]).append(m[y][q]);
      } return r.toString();
   }
   public static void main(String[] z) {
      Scanner sc=new Scanner(System.in);
      System.out.print("Enter the keyword: "); create(sc.nextLine());
      System.out.print("Enter the plaintext: "); String p=prepare(sc.nextLine());
      String e=process(p,true),d=process(e,false);
      System.out.println("\nKey Matrix:");
      for(char[] line:m){for(char c:line)System.out.print(c+" ");System.out.println();}
      System.out.println("\nPrepared text: "+p);
      System.out.println("Encrypted text: "+e);
      System.out.println("Decrypted text: "+d); sc.close();
   }
}
```

OUTPUT

```
Enter the keyword: MONARCHY
Enter the plaintext: INSTRUMENT
Key Matrix:
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z

Prepared text: INSTRUMENT
Encrypted text: GATLMZCLRQ
Decrypted text: INSTRUMENT
```

5. Hill Cipher — Java

CODE

```
import java.util.Scanner;
public class Hill {
    static int[][] key={{3,3},{2,5}}, inv={{15,17},{20,9}};
    static String process(String s,int[][] m) {
        StringBuilder r=new StringBuilder();
        for(int i=0;i<s.length();i+=2) {
            int a=s.charAt(i)-'A',b=s.charAt(i+1)-'A';
            r.append((char)('A'+(m[0][0]*a+m[0][1]*b)%26));
            r.append((char)('A'+(m[1][0]*a+m[1][1]*b)%26));
        } return r.toString();
    }
    public static void main(String[] z) {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter the plaintext: "); String p=sc.nextLine().toUpperCase();
        if(p.length()%2!=0)p+="X";
        String e=process(p,key),d=process(e,inv);
        System.out.println("Prepared plaintext: "+p);
        System.out.println("Encrypted text: "+e);
        System.out.println("Decrypted text: "+d); sc.close();
    }
}
```

OUTPUT

```
Enter the plaintext: MEET
Prepared plaintext: MEET
Encrypted text: WSRZ
Decrypted text: MEET
```

NOTE: Record discrepancy: the PDF's screenshot shows RHHX, but its printed key matrix [[3,3],[2,5]] produces WSRZ for MEET. This program uses the PDF's matrix and prints the mathematically correct WSRZ.

6. DES Algorithm — Java

CODE

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Scanner;
public class DESExample {
    public static void main(String[] args) throws Exception {
        Scanner sc=new Scanner(System.in);
        SecretKeySpec key=new SecretKeySpec("12345678".getBytes("UTF-8"),"DES");
        Cipher c=Cipher.getInstance("DES/ECB/PKCS5Padding");
        System.out.print("Enter Plain Text: "); String msg=sc.nextLine();
        c.init(Cipher.ENCRYPT_MODE,key);
        String enc=Base64.getEncoder().encodeToString(c.doFinal(msg.getBytes("UTF-8")));
        System.out.println("Original Text: "+msg);
        System.out.println("Encrypted Text: "+enc);
        c.init(Cipher.DECRYPT_MODE,key);
        String dec=new String(c.doFinal(Base64.getDecoder().decode(enc)),"UTF-8");
        System.out.println("Decrypted Text: "+dec); sc.close();
    }
}
```

OUTPUT

```
Enter Plain Text: HELLO WORLD
Original Text: HELLO WORLD
Encrypted Text: +dhTwlyrdA35S/6Xk5R+yQ==
Decrypted Text: HELLO WORLD
```

NOTE: Repeatability note: the record uses a randomly generated DES key, which is not included in the PDF. This version uses a fixed key; its ciphertext is therefore repeatable but differs from the record screenshot.

7. Blowfish Algorithm — Java

CODE

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Scanner;
public class BlowfishExample {
   public static void main(String[] args) throws Exception {
      Scanner sc=new Scanner(System.in);
      SecretKeySpec key=new SecretKeySpec("12345678abcdefgh".getBytes("UTF-8"),"Blowfish");
      Cipher c=Cipher.getInstance("Blowfish/ECB/PKCS5Padding");
      System.out.print("Enter Plain Text: "); String msg=sc.nextLine();
      c.init(Cipher.ENCRYPT_MODE,key);
      String enc=Base64.getEncoder().encodeToString(c.doFinal(msg.getBytes("UTF-8")));
      System.out.println("Original Text: "+msg);
      System.out.println("Encrypted Text: "+enc);
      c.init(Cipher.DECRYPT_MODE,key);
      String dec=new String(c.doFinal(Base64.getDecoder().decode(enc)),"UTF-8");
      System.out.println("Decrypted Text: "+dec); sc.close();
   }
}
```

OUTPUT

```
Enter Plain Text: HELLO WORLD
Original Text: HELLO WORLD
Encrypted Text: pRMu/LxU+6DTd19UH1pbgQ==
Decrypted Text: HELLO WORLD
```

NOTE: Repeatability note: the record uses a randomly generated Blowfish key, which is not included in the PDF. This version uses a fixed key; its ciphertext is therefore repeatable but differs from the record screenshot.

8. Rijndael (AES) Algorithm — Java

CODE

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Scanner;
public class RijndaelExample {
   public static void main(String[] args) throws Exception {
      Scanner sc=new Scanner(System.in);
      SecretKeySpec key=new SecretKeySpec("0123456789abcdef".getBytes("UTF-8"),"AES");
      Cipher c=Cipher.getInstance("AES/ECB/PKCS5Padding");
      System.out.print("Enter Plain Text: "); String msg=sc.nextLine();
      c.init(Cipher.ENCRYPT_MODE,key);
      String enc=Base64.getEncoder().encodeToString(c.doFinal(msg.getBytes("UTF-8")));
      System.out.println("Original Text: "+msg);
      System.out.println("Encrypted Text: "+enc);
      c.init(Cipher.DECRYPT_MODE,key);
      String dec=new String(c.doFinal(Base64.getDecoder().decode(enc)),"UTF-8");
      System.out.println("Decrypted Text: "+dec); sc.close();
   }
}
```

OUTPUT

```
Enter Plain Text: HELLO WORLD
Original Text: HELLO WORLD
Encrypted Text: HLt6tFE//C1tNp/gvtlkXw==
Decrypted Text: HELLO WORLD
```

NOTE: Repeatability note: the record uses a randomly generated AES key, which is not included in the PDF. This version uses a fixed key; its ciphertext is therefore repeatable but differs from the record screenshot.

9. RSA Algorithm — Java

CODE

```
import java.util.Scanner;
public class rsa {
   static long pow(long b,long e,long n) {
      long r=1; b%=n;
      while(e>0){if((e&1)==1)r=r*b%n;b=b*b%n;e>>=1;}
      return r;
   }
   public static void main(String[] z) {
      Scanner sc=new Scanner(System.in);
      System.out.print("Enter p: "); int p=sc.nextInt();
      System.out.print("Enter q: "); int q=sc.nextInt();
      int n=p*q,phi=(p-1)*(q-1);
      System.out.print("Enter e: "); int e=sc.nextInt();
      int d=1; while((d*e)%phi!=1)d++;
      System.out.print("Enter message: "); int m=sc.nextInt();
      long c=pow(m,e,n),dec=pow(c,d,n);
      System.out.println("n = "+n); System.out.println("phi(n) = "+phi);
      System.out.println("Public Key: ("+e+", "+n+")");
      System.out.println("Private Key: ("+d+", "+n+")");
      System.out.println("Encrypted Message: "+c);
      System.out.println("Decrypted Message: "+dec); sc.close();
   }
}
```

OUTPUT

```
Enter p: 61
Enter q: 53
Enter e: 17
Enter message: 65
n = 3233
phi(n) = 3120
Public Key: (17, 3233)
Private Key: (2753, 3233)
Encrypted Message: 2790
Decrypted Message: 65
```

10. Diffie-Hellman Key Exchange — Java

CODE

```
import java.util.Scanner;
public class DiffieHellmankeyexchange {
   static long pow(long b,int e,int p) {
      long r=1; b%=p;
      while(e>0){if((e&1)==1)r=r*b%p;b=b*b%p;e>>=1;}
      return r;
   }
   public static void main(String[] z) {
      Scanner sc=new Scanner(System.in); int p=23,g=5;
      System.out.print("enter private key of a:"); int a=sc.nextInt();
      System.out.print("enter private key of b:"); int b=sc.nextInt();
      long pa=pow(g,a,p),pb=pow(g,b,p),ka=pow(pb,a,p),kb=pow(pa,b,p);
      System.out.println("Public prime "+p); System.out.println("Public generator "+g);
      System.out.println("Alice public key is "+pa); System.out.println("Bobs public key is "+pb);
      System.out.println("Shared key of Alice "+ka); System.out.println("Shared key of Bob "+kb);
      System.out.println(ka==kb?"Key exchange successful!":"Key exchange failure"); sc.close();
   }
}
```

OUTPUT

```
enter private key of a:6
enter private key of b:15
Public prime 23
Public generator 5
Alice public key is 8
Bobs public key is 19
Shared key of Alice 2
Shared key of Bob 2
Key exchange successful!
```

11. SHA-1 Message Digest — Java

CODE

```
import java.security.MessageDigest;
import java.util.Scanner;
public class SHA1Example {
   public static void main(String[] z) throws Exception {
      Scanner sc=new Scanner(System.in);
      System.out.print("Enter the text: "); String msg=sc.nextLine();
      byte[] hash=MessageDigest.getInstance("SHA-1").digest(msg.getBytes("UTF-8"));
      StringBuilder hex=new StringBuilder();
      for(byte b:hash)hex.append(String.format("%02x",b&0xff));
      System.out.println("Original Message: "+msg);
      System.out.println("SHA-1 Message Digest: "+hex); sc.close();
   }
}
```

OUTPUT

```
Enter the text: Hello
Original Message: Hello
SHA-1 Message Digest: f7ff9e8b7bb2e09b70935a5d785e0cc5d9d0abf0
```

12. MD5 Message Digest — Java

CODE

```
import java.security.MessageDigest;
import java.util.Scanner;
public class MD5Example {
   public static void main(String[] z) throws Exception {
      Scanner sc=new Scanner(System.in);
      System.out.print("Enter the text: "); String msg=sc.nextLine();
      byte[] hash=MessageDigest.getInstance("MD5").digest(msg.getBytes("UTF-8"));
      StringBuilder hex=new StringBuilder();
      for(byte b:hash)hex.append(String.format("%02x",b&0xff));
      System.out.println("Original Message: "+msg);
      System.out.println("MD5 Message Digest: "+hex); sc.close();
   }
}
```

OUTPUT

```
Enter the text: Hello
Original Message: Hello
MD5 Message Digest: 8b1a9953c4611296a827abf8c47804d7
```